

Lesson flow — what we store and how it runs

Kidsverse+ — screens 14, 16 and 17

Scope Discover → Spot the Mistake → Mission Complete Date 8 September 2026

The point, in one paragraph

These three screens already produce the most valuable data in the product, and currently throw all of it away. They record whether the child got the answer right and award XP. They do not record that the child needed two hints, got it wrong once, and had to switch from the pizza to the number line before it clicked. **That second sentence is the actual learning signal**, and it is what the Parent Zone and Nova’s recommendations are pretending to know today.

1. What the three screens actually do

Screen	Child can do	What the code keeps
14 — Discover Sharing Supersnack	Read the model, reveal up to 3 hints one at a time, open an easier example, continue	Nothing Hint count and the easier-example flag are React state and die on unmount.
16 — Spot the Mistake Are these four parts equal?	Answer yes/no, reveal hints, press “Show it another way” to swap between pizza, chocolate bar and number line	XP only +10 XP on the first correct answer. Wrong attempts, hints used and every model switch are discarded.
17 — Mission Complete	Read the result, continue	XP + station +20 XP and the journey marker moves. The 80% ring and the two 6/8 skill bars are literals in the JSX.

Concretely, what is lost per attempt

A child who answers correctly first time with no hints, and a child who answers wrong twice, opens all three hints and cycles through all three representations, produce **byte-for-byte identical state** today: +10 XP. Those are two very different children and the app cannot tell them apart.

2. What to store

Three layers, and they should not be confused with each other. Content is authored and read. Events are written and never edited. Everything a parent or Nova reads is derived from events and can be thrown away and rebuilt.

Layer 1 — Content (authored, read-only at runtime)

All of this exists today as module constants in the screens. Moving it to the database is a straight lift, and it is what lets a new mission ship without a deploy.

```

topics          id, subject_id, name, grade, board, order
missions        id, topic_id, title, blurb, est_minutes, xp_award, order
mission_steps   id, mission_id, ordinal, kind          -- discover | practice | complete
step_models     id, step_id, model_key, title          -- pizza | bar | number_line
step_hints      id, step_id, model_key, ordinal, body  -- hints differ per model today
step_questions  id, step_id, prompt, kind, options, answer
skills          id, topic_id, name                    -- "equal parts", "visual fractions"
step_skills     step_id, skill_id                      -- which step evidences which skill

```

Note `step_hints` is keyed by model, not just by step. Spot the Mistake already carries a different hint set for the pizza, the bar and the number line, and that is correct — “check the crust” means nothing on a number line.

Layer 2 — Events (append-only, the part that matters)

```

mission_runs    id, child_id, mission_id, started_at, finished_at,
                xp_awarded, outcome
step_events     id, run_id, step_id, ordinal, at, type, payload (json)
                UNIQUE (run_id, ordinal)

```

event type	payload	emitted when
<code>step_opened</code>	—	the step mounts
<code>hint_revealed</code>	<code>{ index, model_key }</code>	a hint is tapped open
<code>model_switched</code>	<code>{ from, to }</code>	“Show it another way”
<code>easier_example_opened</code>	—	the Easier Example button
<code>answer_submitted</code>	<code>{ value, correct, attempt_no }</code>	yes/no is chosen
<code>step_completed</code>	<code>{ ms_spent }</code>	Continue is pressed

Why append-only, and why an ordinal

These are children on tablets with unreliable wifi. Events queue locally and flush when the network returns, so a lesson finished on a train is not lost. `UNIQUE (run_id, ordinal)` makes the flush idempotent: replaying the queue after a timeout cannot double-count a hint or pay the XP twice. Nothing is ever updated in place, so a bad aggregation is a recomputation, not a data loss.

Layer 3 — Derived (rebuildable, read by the Parent Zone and Nova)

```

child_skill_scores  child_id, skill_id, score, out_of, updated_at
child_topic_mastery child_id, topic_id, mastery, evidence_count, updated_at
child_recommendations child_id, topic_id, reason_code, generated_at

```

These are exactly the three numbers the screens currently invent: the two `6/8` skill bars on screen 17, the mastery ring, and the “Fractions — word problems need support” line in the Parent Zone. Nothing writes to them directly; they are a function of the event log.

3. How it runs

```
screen ---emit(event)--> local queue (IndexedDB)
                          |
                          | flush, batched, idempotent
                          v
                          POST /runs/:id/events
                          |
                          v
                          step_events (append only)
                          |
                          | on step_completed
                          v
                          scoring job --> child_skill_scores
                                          child_topic_mastery
                                          child_recommendations
                                          |
Parent Zone / Nova <-----+ (read only)
```

The scoring rule, concretely

Worth arguing about, but it should be written down rather than left to a screen to invent:

```
step_score = clamp(
  (correct ? 1 : 0)
  - 0.15 * hints_revealed
  - 0.10 * (attempts - 1)
  - 0.10 * model_switches,
  0, 1)

skill_score      = sum of step_score over steps tagged with that skill
topic_mastery    = weighted moving average of the last N step_scores,
                  recent steps weighted higher
```

A right answer that needed three hints and two representation switches scores 0.35, not 1. That is the difference between “knows it” and “got there”, and it is the whole reason for collecting the events.

4. Why this shape and not something simpler

Question	Answer
Why not just store a score per mission?	Because a score cannot answer the only question a parent asks: <i>where is my child stuck?</i> “Correct” and “correct after three hints and a switch to the number line” are the same score and completely different situations. The second one tells us the concept landed but the pizza representation did not — which is a teaching instruction, not a grade.
Why events rather than counters?	Counters answer only the questions you thought of when you wrote them. We have already changed our minds twice about what to show a parent. An event log lets the next question be answered by re-running the aggregation over history rather than by waiting months for fresh data.

Why is content in the database?	Every hint, model and question is a JS constant today, so adding a mission is a code change and a deploy. It also blocks the obvious next step: per-grade and per-board variants of the same mission.
Does this fit the app we have?	Yes, unusually well. The client is already a reducer over dispatched actions (<code>GameProvider</code>). These events are those actions, persisted instead of discarded. The change at each screen is one <code>emit()</code> call where a <code>useState</code> currently sits.
How much per child?	Roughly 8–15 events per mission step, a few hundred bytes each. A child doing three missions a day generates well under a megabyte a year. This is not a volume problem.

5. What I would build first

1. **The event table and `emit()`**. Nothing else. Ship it writing events while the screens keep rendering their current numbers. A week of real event data before anyone depends on it.
2. **The scoring job and the two derived tables.** Now screen 17's ring and skill bars come from the child instead of from a literal.
3. **Content into the database.** Lift the existing constants first, unchanged, so the migration is provably a no-op before any new mission is authored against it.
4. **Recommendations.** Last, because it is the only part that needs the history to exist before it can say anything true.

One thing to decide early

Whether a mission run is resumable. If a child closes the tablet mid-mission and comes back, do they resume the run or start a new one? It changes `mission_runs` and it is much cheaper to settle now than after there is data.